

BOSTIn:
Broad Overview of Sequence and Topology
Incongruence
User-Friendly Phylogenetic Artifact Identification
for Nucleotide and Amino Acid Datasets



James F. Fleming and Torsten Hugo Struck

Table of Contents

Welcome to BOSTIn	3
The Structure of this Manual	3
What is BOSTIn?	3
The GTR model, assumptions and SRH violations	4
Introducing the BOSTIn tests: LB-Score	7
Introducing the BOSTIn tests: nRCFV	8
Introducing the BOSTIn tests: C-Value	10
Introducing the BOSTIn tests: DE-Score	11
Installing and Running BOSTIn	13
Running BOSTIn in your web browser	13
Running BOSTIn on your computer	13
Prerequisites	13
Making BOSTIn work	13
Understanding BOSTIn	15
Understanding Branch Length Heterogeneity: LB-Score	15
Understanding Compositional Heterogeneity: nRCFV	18
Understanding Site Saturation in Nucleotide Datasets: C-Value	22
Understanding Site Saturation in Amino Acid Datasets: DE-Score	25
BOSTIn Genewise	28
What is Genewise?	28
Installing and Running Genewise	28
Understanding The Outputs of Genewise	29

Welcome to BOSTIn

The Structure of this Manual

BOSTIn's manual is structured a little differently to the documentation of other bioinformatics software you might have interacted with before. The reason for this is that we wanted to provide a manual that felt like an expression of the ideas that motivated BOSTIn itself — a comprehensive software for phylogenetic artifact identification that is accessible, but also gives users the tools they need to take that knowledge to learn how to interpret their own data, and to understand why their data looks the way it does.

This manual is split into four sections. The first section is this brief introduction, which also introduces why phylogenetic artifacts are important, and how they occur. We also include, for each of BOSTIn's metrics, a breakdown of how each metric is calculated, and how it can measure the model violation it measures. The second section is a simple explanation of how to run and install BOSTIn — the same as the text you find on the GitHub README. The third section is a more detailed explanation of what all the outputs of BOSTIn really mean, with examples for each of the histogram outputs, so that when you look at your unique data, you'll be able to make clear interpretations, guided by the narrative report. The fourth section then extends this idea to BOSTIn's Gene-Wise function. The fifth and final section is a set of three teaching exercises, each revolving around a different model dataset, so that BOSTIn can be readily deployed in a bioinformatics class or course with minimal preparation.

What is BOSTIn?

BOSTIn means Great, Good, Brilliant or Wonderful in the dialect of my hometown, Birmingham, UK (Figure 1). But in this context, it really means "Broad Overview of Sequence and Topology Incongruence" — because I wanted to find a good way to make a phylogenetics software called BOSTIn. This also fits because, to be blunt, the purpose of BOSTIn is to tell you whether your alignment is high quality or not — whether it's Bostin'.

BOSTIn also takes only a multiple sequence alignment as input, so is reasonably easy and computationally efficient to act as a starting point for an analysis — no prior phylogenies needed! BOSTIn tests your alignment using up to three different tests, based on your specifications. These tests examine the branch length heterogeneity, compositional heterogeneity and entropic site saturation of the dataset, but if any of those terms are new to you, don't worry! BOSTIn outputs a "Narrative Report" that identifies problematic sequences or character states, and more importantly, tells you a little bit about why they might be problematic. In addition, this manual will explain a little bit more about what each metric used by BOSTIn does, what it's actually testing for, and even offer some examples. By the end of this book, you should be well on your way not just to a better understanding of your dataset, but also how phylogenies are generated!



Figure 1) Birmingham's famous Gas Street Basin

The GTR model, assumptions and SRH violations

"You put rubbish in, you get rubbish out"

This was how I was introduced to understanding phylogenetic artifacts by my PhD supervisor, and it remains a pretty sensible, and true, piece of advice. But how you determine what rubbish is — that is a much thornier problem!

The first problems that some phylogenies run into are incorrect orthology assignment and contamination. These problems can generally be referred to as "Biological Incongruence". Basically, sometimes the sequences you think you have aren't the sequences you actually have. This can be because they belong to another organism that was sequenced alongside your target (contamination), or because the history of your target sequence in your target organism was more complicated than you previously realised (either due to multiple duplications and losses, horizontal gene transfer or hybridisation). BOSTIn can't really help with this — it's very much its own region of concern. I'd advise checking out Jesus Lozano-Fernandez's review "A Practical Guide to Design and Assess a Phylogenomic Study", and which explores that subject in more detail in a very approachable way, and offers a range of resources to help.

BOSTIn works on the other kind of disruptive data in multiple sequence alignments — "methodological incongruence". Methodologically incongruent data is data that doesn't fit well with our model. The model of evolution that we generally use to reconstruct evolution is called the General Time Reversible (GTR) model. There is a specific model called the General Time Reversible model in most phylogenetic software, but also almost every other selectable model, from JC69 and JTT through to WAG and C60, is a "special instance" of a GTR model, meaning that if you look at the mathematics of the model it operates in the same way. Like all models, our model of evolution isn't perfect, and it makes a series of mathematical assumptions about

what the sequences in the dataset ought to look like. When those assumptions are violated — they make an ass out of you and me.

There are three assumptions that are key to methodological incongruence — Stationarity, Reversibility and Heterogeneity. Together, these are called the "SRH assumptions". Stationarity is the assumption that the rate of change of amino acids or nucleotides is the same across the dataset. Broadly, that the speed of evolution is constant. This can be violated in a lot of ways — selective bottlenecks, strong variation in generation times and rapid diversification of certain clades can all cause datasets to violate this assumption. Violations of stationarity can cause branch length heterogeneity and long branch attraction — where "long branches" on a phylogeny can end up moving closer to one another, as they do not strongly resemble anything else in the dataset. This is identified by the LB-Score — the Long Branch Score.

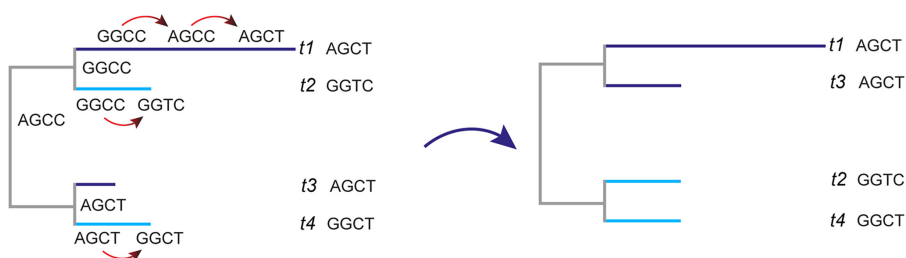


Figure 2) Long Branch Attraction (original figure from Fleming et al 2023). The real evolutionary history of this scenario is shown on the left, and the tree recovered by the alignment on the right. Notice how the many changes in t1 causes it to superficially resemble t3, resulting in a topology distinct from the true tree.

Reversibility, meanwhile, means that the process of amino acid or nucleotide change is "undirected". This means that, for example, the probability of an A changing to a T is the same as the probability of a T changing to an A. This can be violated by directional change — sometimes, due to neofunctionalisation, or bottlenecking, or adaptation, certain nucleotide substitutions are strongly favoured over the reverse substitution. Finally, Homogeneity is the idea that, overall, the composition of nucleotides and amino acids in the dataset remains constant over time. What this means is that, when the tree is being constructed by a tree-building algorithm, although nucleotides and amino acids might change from one to another over time, at all points in time, that the broad overall make-up of the data will be reasonably similar (say 25% of A, 40% T, 10% C and 25% G), whether you are at the root of the tree or its tips. This can sometimes be violated in cases of directed change in the tree — if suddenly, a change in the selective environment means AT is favoured over CG in comparison to other parts of the tree. These are both identified by nRCFV — particularly by its character and taxon-specific values.

Finally there is entropic saturation. This overlies all the assumptions, and can cause, or be caused, by any of them. This is where there has been so much change across parts of the dataset that any historical signal is lost, and replaced only by noise. This is identified by the C-Value in nucleotide data and the DE-Score in amino acid data.

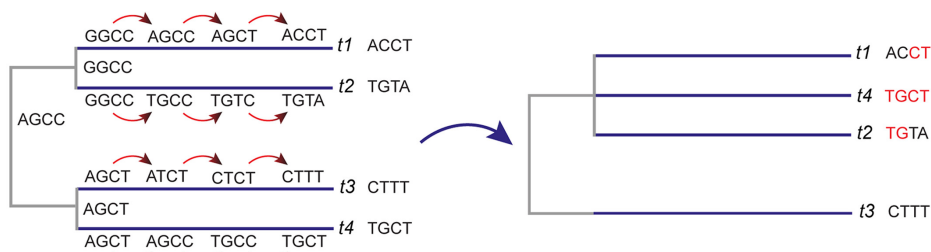


Figure 3) Site Saturation (adapted from Fleming et al 2023). The real evolutionary history of this scenario is shown on the left, and the tree recovered by the alignment on the right. Note how the many changes across all branches have resulted in a final topology that is challenging to reconcile, as these related sequences no longer resemble one another. The artefactual signal (highlighted in red) favours a polytomy of *t1*, *t2* and *t4* together, to the exclusion of *t3*.

How each of these tests works, the mathematics behind them and the reasoning for why they identify the artifacts they identify are explained in more detail in the following section. Each test explanation will follow the same format, giving you the equation first, then breaking it down, then explaining how it measures what it measures.

Introducing the BOSTIn tests: LB-Score

The LB in LB Score stands for Long Branch, and it calculates the heterogeneity of branch lengths in a dataset. We use heterogeneity here, because it detects tips with peculiarly short branches and peculiarly long ones. It uses something called the "Patristic Distance" to calculate the distance between taxa on a tree, which is the sum of the branch lengths connecting two tips on a tree. To calculate it, you navigate from tip 1 to tip 2 by going from tip 1, down in the tree to the node that connects them, then up to tip 2, and summing the branches you travelled along. The LB-Score equals:

$$LB_i = \left(\frac{PD_i}{PD_a} - 1 \right) \times 100$$

Where LB_i is equal to the LB Score for a particular tip (tip i), PD_i is equal to the average patristic distance of that tip to all other tips, and PD_a is equal to the patristic distances of the entire dataset. This equation might look intimidating, but what it really does (with the -1 and then multiplying everything by 100) is calculate the *percentage deviation of branch lengths for a given tip*. That is, the difference of this specific tip to all other tips in the dataset as a percentage of the average of all the distances between every set of two tips in the entire dataset.

We can then use the standard deviation of these tip-specific values to produce an LB-Score that reflects the entire dataset. The standard deviation is used here as it allows us to quantify how much variation there is in branch lengths within the dataset into a single number. This allows us not just to be able to identify which particular tips in our dataset are outliers from an average, but also whether our dataset has more variation in branch lengths when compared to another dataset.

This means that the LB-Score produces tip-specific values as well as a whole dataset summary. This can be useful when you want to identify particular tips that you think might be causing problems in your dataset. We'll talk more about how to interpret those results in the Understanding BOSTIn section below.

You might also notice that BOSTIn claims to need no topology, but then we introduce the LB-Score, which clearly needs one! BOSTIn uses a Neighbour Joining tree to generate this value, and this is generally reliable, even when the actual Neighbour Joining tree is wildly inaccurate! The reason for this is that we aren't actually interested in the topology of the tree — we're interested in the relative distribution of branch lengths within it. These might change if the data is analysed using a more complicated model, but long branches are long branches, and if anything, using a less complex neighbour joining method makes them easier to identify.

Introducing the BOSTIn tests: nRCFV

nRCFV stands for normalised Relative Compositional Frequency Value. It measures the amount of heterogeneity in a DNA or amino acid dataset — the composition of nucleotides and amino acids relative to the other nucleotides and amino acids in the dataset. nRCFV equals:

$$nRCFV = \frac{\sum_{i=1}^n \sum_{j=1}^m \frac{|\mu_{ij} - \mu_j|}{n}}{p^{-0.5} * n^{0.01} * c * 100}$$

This might be the most intimidating equation in BOSTIn, but it is deceptively simple. The calculation of nRCFV can be viewed as a series of steps:

1. Find the frequency (a percentage expressed as 0-1, so 20%, for example, would be 0.2) of a particular character in a sequence in a dataset — μ_{ij}
2. Work out the difference between the frequency of this character in this sequence and the average frequency of that same character across the whole dataset — μ_j .
IMPORTANT NOTE: || is a way of signalling that you are interested in the *absolute difference*. So, if the character's frequency in the sequence is 20% (0.2), and the average frequency across the entire dataset is 30% (0.3), then $|\mu_{ij} - \mu_j| = 0.1$, which is the same as if the character's frequency in the sequence was 30% (0.3), and the average frequency across the entire dataset was 20% (0.2).
3. Divide the difference between the frequency of this character in this sequence and the average frequency of that same character across the whole dataset by how many tip are in the alignment — n .
4. Repeat that for every character and sum it — $\sum_{j=1}^m$.
5. Repeat *that* for every tip in the alignment — $\sum_{i=1}^n$. What we have done by this point is calculate the Relative Compositional Frequency Value - the "RCFV" bit of nRCFV. So all we have to do is normalise it!
6. Divide the resulting number by a "normalisation constant" — $p^{-0.5} * n^{0.01} * c * 100$. Here, $p^{-0.5}$ is the number of positions in the dataset — the length of the alignment — to the power of -0.5, $n^{0.01}$ is the number of tips in the dataset to the power of 0.01 and c is the number of different characters in the dataset (so that is 4 for nucleotides and 20 for amino acids). Then you multiply all that by 100 to make the number look a bit nicer.

Step 6 is the bit that makes nRCFV distinct from RCFV — an older metric that is the "parent" of nRCFV. This normalisation constant removes the bias that the size of the alignment has on RCFV, which allows you to compare alignments that are otherwise very different — long genes to short genes, or alignments with lots of taxa to alignments with very few taxa.

What this broadly means is that a large nRCFV means that the alignment is quite heterogeneous and might be vulnerable to topological artifacts caused by compositional heterogeneity. Meanwhile, a low nRCFV means that your data is probably quite homogeneous, and is unlikely to cause problems.

nRCFV also comes in two additional flavours: csnRCFV and tsnRCFV. csnRCFV is "character-specific" and tsnRCFV is "taxon-specific". This means that they can instead tell you if one particular nucleotide or amino acid is causing a problem in your dataset (look for a character with a particularly high csnRCFV) or if a particular tip in your dataset is instead responsible for problems (a tip with a particularly high tsnRCFV). We'll explain more about how to interpret all of these values in the Understanding BOSTIn section, with some examples.

Introducing the BOSTIn tests: C-Value

The Convergence Value (C-Value) is a measure of entropic saturation. It is one of the two BOSTIn tests for entropic saturation, since each are designed for a specific type of data - C-Value is for nucleotide (DNA) datasets. The C-Value equals:

$$C = \frac{\sigma\left(\frac{ti}{tv}\right)}{\sigma\left(\frac{n_d}{n}\right)}$$

The C-value seems to be the simplest of the equations in BOSTIn, but may be conceptually one of the more challenging to understand, because it relies on standard deviations – measuring how far points in the dataset are distributed from the averages. The ratio $\frac{ti}{tv}$ is the transition-transversion ratio. A transition is when a purine changes to another purine, or a pyrimidine changes to another pyrimidine (for example A to T or G to C). A transversion, meanwhile, is when a purine changes a pyrimidine, or vice versa (for example A to G or C to T). This is an important way to measure entropy in a nucleotide dataset, as we expect transitions to be more common than transversions, as transversions can often affect the amino acids that are transcribed by a codon. To calculate this, BOSTIn makes pairwise comparisons for each site in the dataset, which produces a transition-transversion ratio for each site in the dataset.

A $\frac{ti}{tv}$ ratio of 1 is generally assumed to indicate total uninformative entropic saturation, and ideally this value should be as high as possible, representing many transitions and only a few transversions. The lower-case sigma, σ , represents the standard deviation of these ratios.

The standard deviation of the transition-transversion ratio is then divided by the standard deviation of the uncorrected genetic distances: $\frac{n_d}{n}$. An uncorrected genetic distance (sometimes also called a *p*-distance) is how many sites differ between the two sequences (n_d) divided by the length of those sequences (n). As it counts both a transition and a transversion as 1 change, it helps balance the metric by detecting raw change. Very high uncorrected genetic distances also tend to indicate high saturation in a dataset - while transitions ought to be more common than transversions, if huge numbers of any kinds of changes are still occurring, it can still obscure any phylogenetic information. When the C-Value is 10 or less, then, that tends to mean that both $\sigma\left(\frac{ti}{tv}\right)$ and $\sigma\left(\frac{n_d}{n}\right)$ are probably quite high, which indicates that the dataset is saturated. When the C-Value is less than 20, that might be cause for concern, and it means you should monitor the sequences with those low C-Values more closely. If C-Value is greater than 20, they probably aren't particularly saturated!

Commented [1]: Or RNA?

Introducing the BOSTIn tests: DE-Score

Just like the C-Value, the Dayhoff Exchange Score (DE-Score) is a measure of entropic saturation. Both are available in BOSTIn because each one is designed for a specific type of data - DE-Score is for amino acid (protein) datasets. The DE-Score equals:

$$DEScore = \frac{(\frac{1}{n} \sum_{t=1}^n \sum_{i=1}^{n-1} \frac{Cw_{it}}{Cb_{it} + Cw_{it}}) - 0.177}{0.255n^{-0.15}}$$

Similar to nRCFV, this equation looks complex, but we can break it down to be surprisingly simple. First, though, we should talk about what Dayhoff categories are, and what Dayhoff Exchange means. The 20 amino acids that form the amino acid alphabet can be broken down into distinct categories based on their physical and chemical properties. One of the most popular ways to group amino acids is called "Dayhoff 6", defined by Margaret Dayhoff and Richard Eck in 1966. Dayhoff 6 groups amino acids into 6 categories — Small, Hydrophobic, Acid-Amide, Basic, Aromatic and Sulfur Polymerisation. These each contain different numbers of amino acids, too: Small comprises 5 amino acids, Hydrophobic and Acid-Amide both comprise 4, Basic and Aromatic 3 and Sulfur Polymerisation only 1 (Cysteine).

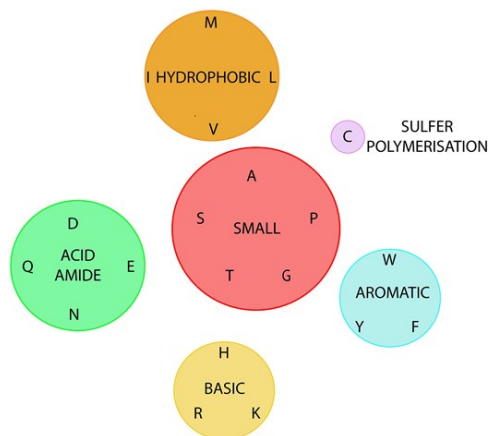


Figure 4) The twenty amino acids divided into the 6 Dayhoff categories (adapted from Fleming & Struck 2026). The six categories are Small, Hydrophobic, Acid-Amide, Basic, Aromatic and Sulfur Polymerisation.

Just like with a $\frac{ti}{tv}$ ratio (explained in the C-Value explanation above), that means that we can create a ratio of within category changes vs. between category changes. Between category changes change the properties of the amino acid, which could change the structure or function

of the protein, so these should be rarer than within category changes which keep the properties relatively similar.

The frequency of Within/Between Category changes is this part of the DE-Score Equation: $\frac{Cw_{it}}{Cb_{it}+Cw_{it}}$, where Cw_{it} is the Within Category Changes within a sequence, and Cb_{it} is the Between Category change. When $\frac{Cw_{it}}{Cb_{it}+Cw_{it}}$ is equal to 1, it means that there are only within category changes, and when it is 0, it means that there are only between category changes. We then calculate this value for each tip in the dataset and sum it together, then average it: $\frac{1}{n} \sum_{t=1}^n \sum_{i=1}^{n-1}$. Taking this average Within Category change frequency, we then subtract 0.177. This sounds like a strange value, but a Within Category change frequency of 0.177 actually represents total saturation in a way that it becomes totally uninformative, so this allows our metric to measure from 0.

We then divide all of this by $0.255n^{-0.15}$. As elsewhere in this manual, n represents the number of tips in the dataset. This is also a normalisation constant — as we use pairwise comparisons, there is a chance that the top of the equation ends up biased by the size of the dataset — the number of tips and the length of the sequences. This is because when making a pairwise comparisons, a single change in a small dataset can have a bigger impact than a single change in a larger dataset, as that single change is a larger proportional change. This normalisation constant balances that effect out.

Installing and Running BOSTIn

Running BOSTIn in your web browser

The easiest way to run BOSTIn is to open it up in your web browser, at:

https://jffleming.shinyapps.io/bostin_r/

From there, you can simply upload your alignment file, click the boxes for the analyses you would like to run, and get going right away!

However, you can only assess one multiple sequence alignment file at a time using the web browser, and you can only upload files that are 10MB or less in size, so if your project is a bit more demanding, I would recommend installing BOSTIn on your computer (as below).

Running BOSTIn on your computer

You can find BOSTIn at its Github page (<https://github.com/JFFleming/BOSTIn>). If it is your first time using the Command Line, I recommend checking out this guide here: , which will help familiarise yourself with some of the basic core commands!

Prerequisites

BOSTIn requires Perl and R. Many computers already come bundled with Perl, but if you don't have access to it (which you can find out by typing "which perl" into your command line terminal), you can install it here: <https://www.perl.org/get.html> To install R, a download link can be found here: <https://cran.r-project.org/>

In addition, BOSTIn requires one R package (Phangorn). Once you have installed Perl and R, you can install that package with this command:

```
Rscript -e 'install.packages("phangorn", repos="https://cloud.r-project.org")'
```

Making BOSTIn work

Once you have the prerequisites installed, then BOSTIn will work provided you have all of the BOSTIn scripts in the same directory as the Fasta file. However, moving them around your system every time you want to do an analysis is a pain, so this next step turns BOSTIn into an executable you can use anywhere on your computer! First, place the BOSTIn directory in the place you want to keep it. Use

```
pwd
```

to find the complete address of that directory and copy that address.

In your home directory (~/), you will find a file called either .bashrc or .bash_profile. Add this line to the bottom of that file, where <pwd address> is the address you copied earlier.

.

```
export PATH=$PATH:<pwd address>/BOSTIn/BOSTIn_v1_3
```

This makes sure that the next time you open your terminal, it registers the BOSTIn directory as a place to look for executables. You will need to close the terminal and open it again for the change to kick in. Finally, go inside the BOSTIn_v1_3 directory (where all the scripts are) and run the following command:

```
chmod u+x+r *
```

This elevates these files to make sure they can do what you need them to do. If you are on a Mac, you might also need to run this

```
xattr -d com.apple.quarantine *
```

Which removes the "quarantine" status from the software package - basically telling your mac it isn't harmful.

Then, you should be ready to go! If you're having trouble installing BOSTIn, please don't hesitate to get in touch through the Github (<https://github.com/JFFleming/BOSTIn>)!

Understanding BOSTIn

BOSTIn produces a lot of output files, and a lot of numbers, and not all of them are intuitive to grasp at first. This chapter aims to demystify the results in BOSTIn beyond the guidance supplied by the narrative report, show some of the shapes of the BOSTIn histograms, and describe what these mean in practice. For most cases, the narrative report of BOSTIn should tell you what you need to know — it will describe the distribution of your data and recommend some sequences or characters that might be causing problems. The red flag taxa and characters are highly likely to be problematic, while yellow flags might cause problems, but should probably be observed with caution rather than removed.

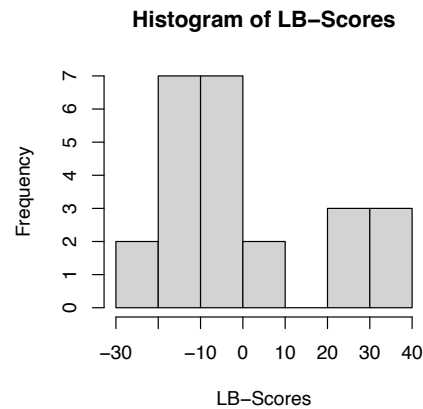
Understanding Branch Length Heterogeneity: LB-Score

BOSTIn produces 6 output files for Branch Length Heterogeneity:

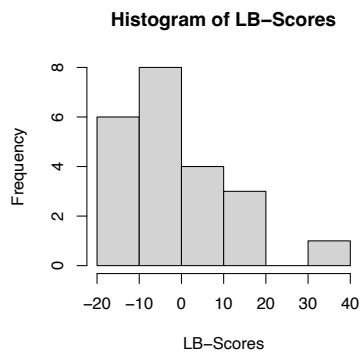
- `<prefix>.LBSummary.txt` — This is a summary of the whole dataset LB-Scores — the mean and quartile values.
- `<prefix>.LBI-scores` — This is a list of the LB-Scores for each taxa.
- `<prefix>.LBSummary.histogram.pdf` — This is a histogram of the taxon-specific LB-Scores, so that you can see what the overall distribution of branch length heterogeneity looks like in your dataset.
- `<prefix>.BranchHeterogeneity.NarrativeReport.txt` — This is the narrative report explaining your Branch Length Heterogeneity results.
- `<prefix>.LBScore.Taxa.redflags.txt` — This is a list of the taxa that BOSTIn identifies as "red flags" in your dataset based on branch length heterogeneity — ones that should be considered for removal or alteration.
- `<prefix>.LBScore.Taxa.yellowflags.txt` — This is a list of the taxa that BOSTIn identifies as "yellow flags" in your dataset based on branch length heterogeneity — ones that could cause problems.

The first place to look is at the LBSummary file. This provides you with a lot of the core numerical information describing your data. The standard deviation of the LB-Scores is the most important — it summarises the LB-Scores across the whole dataset. A high standard deviation of LB-Scores indicates a large amount of branch length heterogeneity (which is not ideal). Also important here are the minimum and maximum. A low minimum with a large range could indicate something called an "umbrella effect", where you have many short branches and some very long branches, while a large range with a minimum close to zero indicates a more typical long branch problem.

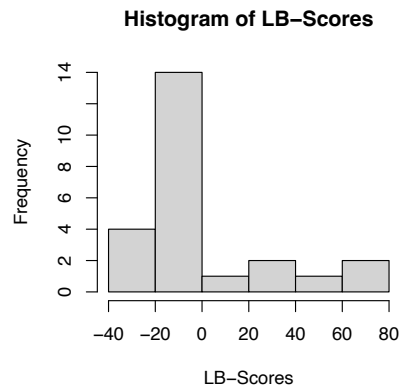
To better understand your data, it's always a good idea to check the histograms. Here, we present 3 likely scenarios that you might see:



A set of LB-Scores like this show a high distribution of LB-Scores towards the right side of the histogram. They are also clearly outliers, as there is a strong gap between LB Score 10 and 20. This indicates several sequences that are liable to be particularly affected by long branch attraction, but also that removing these sequences will likely strengthen the dataset.



This second histogram, meanwhile, represents a much safer dataset. Only a single sequence — a clear outlier — sits outside the broader distribution, which itself is centred around the left side of the graph, trending towards smaller deviations from the mean patristic distance (this is good!). Remove that problematic sequence, and your dataset will likely look a lot nicer.



A long tail, like this, on the other hand, can be more challenging to ameliorate. It can be hard to work out where the Branch Length Heterogeneity begins, and where a normal branch length distribution begins. My recommendation in a case like this would be to remove the two sequences that produce the largest values (in the 60-80 bin), then rerun BOSTIn and observe how the histogram changes. This kind of staged analysis can help you better understand problematic datasets such as this.

Understanding Compositional Heterogeneity: nRCFV

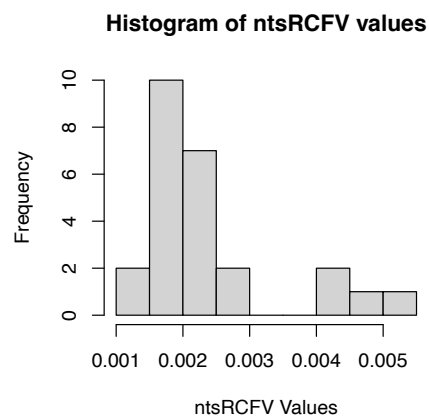
BOSTIn produces 13 output files for Compositional Heterogeneity analyses:

- <prefix>.RCFV.txt — This shows the nRCFV and RCFV of the dataset
- <prefix>.Frequencies.txt — This is a list of the basic frequencies of each amino acid or nucleotide across each taxa in the dataset
- <prefix>.ncsRCFV.summary.txt — This shows a list of summary statistics for the normalised csRCFV scores. The mean and median, alongside the standard deviation and median absolute deviation, help us understand the distribution of compositional heterogeneity across a dataset.
- <prefix>.ncsRCFV.txt — This is a simple list of the normalised csRCFV score for each character in your dataset.
- <prefix>.ncsRCFV.histogram.pdf — This is a histogram of the normalised csRCFV scores, so that you can see what the overall distribution of compositional heterogeneity looks like across characters in your dataset.
- <prefix>.ntRCFV.summary.txt — This shows a list of summary statistics for the normalised tRCFV scores. The mean and median, alongside the standard deviation and median absolute deviation, help us understand the distribution of compositional heterogeneity across a dataset.
- <prefix>.ntRCFV.txt — This is a simple list of the normalised tRCFV score for each taxa in your dataset.
- <prefix>.ntRCFV.histogram.pdf — This is a histogram of the normalised tRCFV scores, so that you can see what the overall distribution of compositional heterogeneity looks like across taxa in your dataset.
- <prefix>.CompositionalHeterogeneity.NarrativeReport.txt — This is the narrative report explaining your CompositionalHeterogeneity results (see narrative reports section below)
- <prefix>.ncsRCFV.CompHet.Character.redflags.txt — This is a list of the characters that BOSTIn identifies as "red flags" in your dataset based on compositional heterogeneity — ones that should be considered for removal or alteration.
- <prefix>.ncsRCFV.CompHet.Character.yellowflags.txt — This is a list of the characters that BOSTIn identifies as "yellow flags" in your dataset based on compositional heterogeneity — ones that could cause problems.

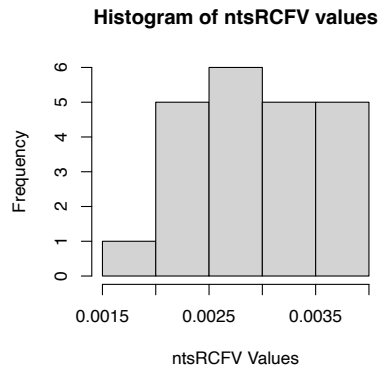
- <prefix>.ntRCFV.CompHet.Taxa.redflags.txt — This is a list of the taxa that BOSTIn identifies as "red flags" in your dataset based on compositional heterogeneity — ones that should be considered for removal or alteration.
- <prefix>.ntRCFV.CompHet.Taxa.yellowflags.txt — This is a list of the taxa that BOSTIn identifies as "yellow flags" in your dataset based on compositional heterogeneity — ones that could cause problems.

The nRCFV in isolation isn't always helpful — it's best used in the context of other values. So first, I'd recommend checking the ncsRCFV and ntsRCFV summary files. For example, if the nRCFV is larger than the median ntsRCFV, that can indicate compositional heterogeneity in the dataset, and so it's best to check the histogram after that.

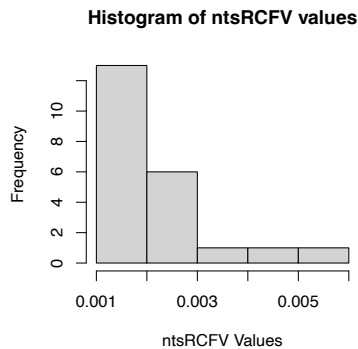
To better understand your data, it's always a good idea to check the histograms. Here, we present 3 likely scenarios that you might see for ntsRCFV:



This represents a more clear-cut scenario. The sequences on the right side of the histogram are compositionally heterogeneous — removing all four of them from the dataset might help with your phylogenetic resolution. If you can't remove them for whatever reason, it might be worth looking at your ncsRCFV values too, to see if recoding the data might help. If you have nucleotide data, another option might be to resample an alignment using only the third codon position and run BOSTIn again to see if you observe the same pattern.

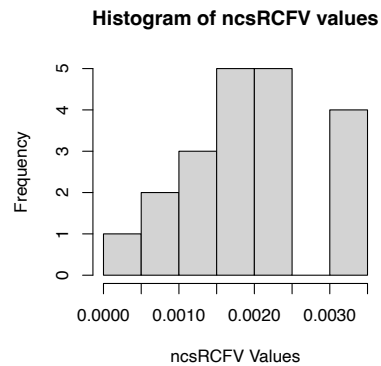


This histogram, meanwhile, represents an almost ideal case (a truly ideal case would be a normal distribution but those are rare in real data). Here, ntsRCFV is distributed relatively evenly around the central point, straying a little more into being heterogeneous than homogeneous, but with no clear outliers. This dataset is probably good to go.

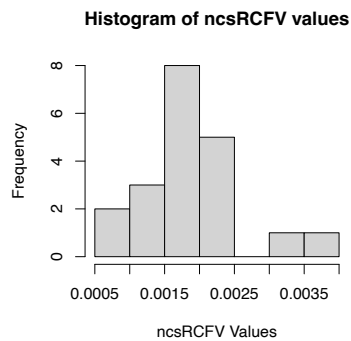


This third scenario represents something more complex. Here, we have a long tail of increasing heterogeneity, but it is difficult to know how distinct this heterogeneity is from the homogenous peak of the dataset. By removing the most heterogeneous sequences (similar to the third scenario in the LB-Score above), and then retesting the sample with nRCFV we might get a better idea of what the internal distribution of the high peak in the leftmost bin looks like. This will help us establish whether it is prudent to remove all three sequences that comprise the tail, or just one or two.

Meanwhile, to better understand ncsRCFV, it's easiest to look at amino acid data first, as there are 20 characters rather than 4 to observe. But all these same principles can still be applied. As ncsRCFV is a little simpler — and has fewer possible distributions — in comparison to the other metrics in BOSTIn, we've just offered 2 example histograms below.



This histogram looks concerning but is actually close to what we'd like to see. There is a reasonably normal-looking distribution of characters across the dataset (if we ignore the gap between 0.0025 and 0.0030), and no strong outliers that might indicate significant over-representation of one character over another.



This, however, looks similar, but is quite concerning. Here, that same gap, followed by only 2 characters, represents a significant outlier from the rest of the distribution (the other 18 characters). Paying attention to the frequency on the y axis is key when interpreting ncsRCFV histograms. Identify which characters these are and then consider whether recoding might help. Combining characters will mean that the average frequency of the new, combined character will be balanced, which can ameliorate problematic heterogeneity in a dataset.

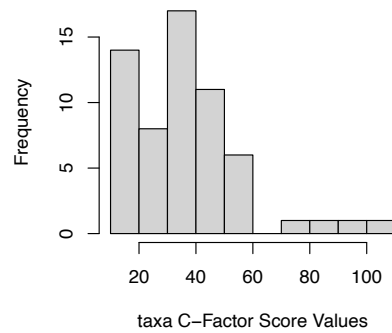
Understanding Site Saturation in Nucleotide Datasets: C-Value

BOSTIn produces 7 output files for Entropic Site Saturation analyses on nucleotide datasets:

- `<prefix>.SiteSaturation.TaxaFrequencies.summary.txt` This shows a list of summary statistics for the taxa-specific C-Values. The mean and median, alongside the standard deviation and median absolute deviation, help us understand the distribution of site saturation across a dataset.
- `<prefix>.SiteSaturation.TotalFrequencies.txt` — This is a file containing the C-Value of your entire dataset.
- `<prefix>.SiteSaturation.TaxaFrequencies.txt` — This is a simple list of the C-Values for each taxa in your dataset.
- `<prefix>.SiteSaturation.TaxaFrequencies.histogram.pdf` — This is a histogram of the taxa-specific C-Values, so that you can see what the overall distribution of site saturation looks like across taxa in your dataset.
- `<prefix>.SiteSaturation.NarrativeReport.txt` — This is the narrative report explaining your Site Saturation results (see narrative reports section below)
- `<prefix>.SiteSaturation.TaxaFrequencies.SiteSat.Taxa.redflags.txt` — This is a list of the taxa that BOSTIn identifies as "red flags" in your dataset based on site saturation — ones that should be considered for removal or alteration.
- `<prefix>.SiteSaturation.TaxaFrequencies.SiteSat.Taxa.yellowflags.txt` — This is a list of the taxa that BOSTIn identifies as "yellow flags" in your dataset based on site saturation — ones that could cause problems.

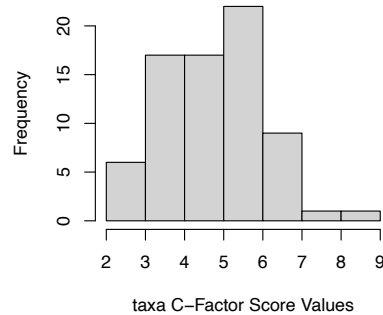
To better understand how saturation is distributed across your dataset, looking at the C-Value histogram can also help you understand your dataset. C-Value is a bit more intuitive to interpret than many other BOSTIn metrics, however. You are looking first at the frequencies below 10 (which represent sequences very high in entropic saturation), and then those below 20 (representing sequences with quite high entropic saturation). Anything above 20 is great.

Histogram of taxa C-Factor Score value:



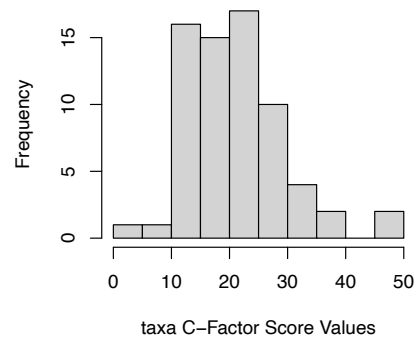
Take, for example, this ideal scenario. Nothing below 10, with scores all the way up to over 100! This is a dataset that is unlikely to be affected by entropic site saturation, despite the strange looking appearance of the distribution in the histogram.

Histogram of taxa C-Factor Score value:



This histogram, meanwhile, presents an absolute worst-case scenario. Not a single score above 10. It is unlikely that this dataset will be particularly informative for phylogenetic analysis, as it appears to be significantly affected by entropic site saturation.

Histogram of taxa C-Factor Score value:



This represents a middle ground between our two extremes. two sequences show values below 10, and around 30 more are in that danger zone between 10 and 20 where it might be worth keeping an eye on where they are placed when you reconstruct a topology. But a good majority of the dataset is higher than 20. I'd recommend removing the two sequences with values below 10, then running your phylogenetic reconstruction.

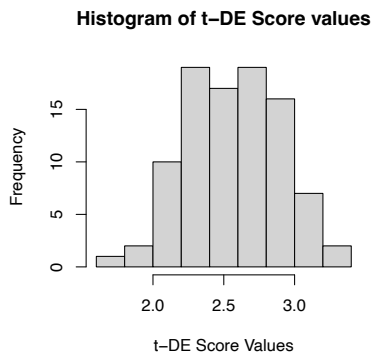
Understanding Site Saturation in Amino Acid Datasets: DE-Score

BOSTIn produces 7 output files for Entropic Site Saturation analyses on amino acid datasets:

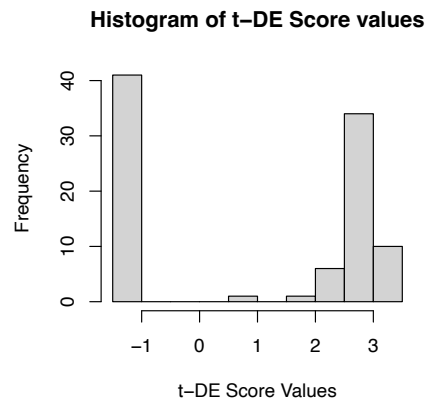
- `<prefix>.SiteSaturation.TotalFrequencies.txt` — This is a file containing the DE-score of your entire dataset.
- `<prefix>.SiteSaturation.TaxaFrequencies.summary.txt` This shows a list of summary statistics for the taxa-specific DE-scores. The mean and median, alongside the standard deviation and median absolute deviation, help us understand the distribution of site saturation across a dataset.
- `<prefix>.SiteSaturation.TaxaFrequencies.txt` — This is a simple list of the DE-scores for each taxa in your dataset.
- `<prefix>.SiteSaturation.TaxaFrequencies.histogram.pdf` — This is a histogram of the taxa-specific DE-Scores scores, so that you can see what the overall distribution of site saturation looks like across taxa in your dataset.
- `<prefix>.SiteSaturation.NarrativeReport.txt` — This is the narrative report explaining your Site Saturation results (see narrative reports section below)
- `<prefix>.SiteSaturation.TaxaFrequencies.SiteSat.Taxa.redflags.txt` — This is a list of the taxa that BOSTIn identifies as "red flags" in your dataset based on site saturation — ones that should be considered for removal or alteration.
- `<prefix>.SiteSaturation.TaxaFrequencies.SiteSat.Taxa.yellowflags.txt` — This is a list of the taxa that BOSTIn identifies as "yellow flags" in your dataset based on site saturation — ones that could cause problems.

When looking at your DE-Score output data, first check the TotalFrequencies file. This gives you a list of the useful DE-Score statistics. Check first to see how close your DaCER (Dayhoff Category Exchange Ratio) is to 0.177. If it is below, the entire dataset is likely highly saturated. This also corresponds to a DE-Score of 0 or less. If your DE-Score is higher than 0, then the data isn't completely saturated, but a DaCER of better than 0.265 is generally what you should be aiming for! If you are looking at Genewise data instead of single gene data, then make sure to only look at the DE-Scores — DaCERs aren't comparable between datasets.

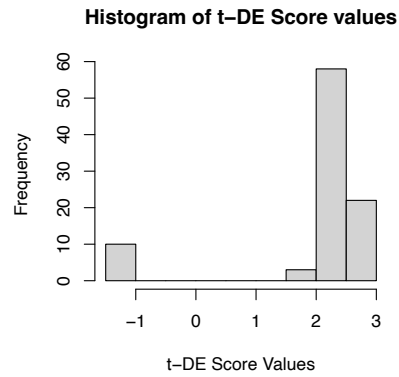
If your DE-Score is lower (and even if it isn't), check the TaxaFrequencies.summary file. This will give you a breakdown of the maximum and minimum DE-Scores per taxa. The same rules apply — you want to find DE-Scores above 0 and DaCERs of 0.265 or better. If you want to explore the patterns in the data, check the histograms. Here, we present 3 likely scenarios that you might see:



This is the ideal scenario — a nice normal distribution around a point considerably higher than the critical DE-Score for the dataset. This suggests a distribution of entropic saturation across the tips in your dataset, but that none of the sequences are particularly strongly affected by it.



This, on the other hand, is a worst-case scenario. A high peak in the left of the graph, with taxon-specific DE-Scores well below 0. This represents a highly entropically saturated dataset — perhaps to an unretrievable degree. Here, discarding entropically saturated sequences would mean removing almost half of the alignment (42), and as the Dayhoff score measures Dayhoff Category Exchanges as a proxy for entropic saturation, recoding is unlikely to be able to assist. It might just be a particularly phylogenetically uninformative gene.



Most more negative results, however, will likely look more like this, with a strong cluster of unsaturated sequence sin the right of the histogram, and some saturated sequences in the left. 10 sequences in this dataset are significantly saturated with respect to the remaining 83. It might be a good idea to prune these saturated sequences away and see how the resulting tree reconstruction differs.

BOSTIn Genewise

What is Genewise?

Genewise is a special version of BOSTIn intended for large, multi-gene datasets. Have you ever had hundreds of genes across your study taxa, and not been sure which genes are best to select for your final analysis? Genewise aims to address this problem. All it needs is a file with a list of the multiple sequence alignments you'd like to analyse (and for them to all be in the same folder in your computer). Then watch it go!

Gene-Wise presents an alternative way to use BOSTIn for multiple gene datasets. Gene-Wise is a script, written in bash, that acts as a way to easily automate multiple BOSTIn analyses in order to efficiently analyse multiple genes in a single batch. Then, it outputs a series of files for each BOSTIn metric selected that analyses the genes in the context of one another, helping you pick the most sensible genes for your next multi-gene alignment dataset.

Installing and Running Genewise

Once BOSTIn is installed, following the instructions above, install Gene-Wise BOSTIn using the same process. First, place the BOSTIn Gene-Wise directory in the place you want to keep it. Use

```
pwd
```

to find the complete address of that directory and copy that address.

In your home directory (~), you will find a file called either `.bashrc` or `.bash_profile`. Add this line to the bottom of that file, where `<pwd address>` is the address you copied earlier.

```
export PATH=$PATH:<pwd address>/BOSTIn_v1_0/GeneWise
```

This makes sure that the next time you open your terminal, it registers the BOSTIn directory as a place to look for executables. You will need to close the terminal and open it again for the change to kick in. Finally, go inside the GeneWise directory, with all the scripts, and run the following command:

```
chmod u+x+r *
```

This elevates these files to make sure they can do what you need them to do. If you are on a Mac, you might also need to run this:

```
xattr -d com.apple.quarantine *
```

Which removes the "quarantine" status from the software package — basically telling your mac it isn't harmful.

Then, you should be ready to go!

BOSTIn Genewise uses the same format as BOSTIn, with the same three options — `--blh` for branch length heterogeneity, `--s` for site saturation and `--ch` for compositional heterogeneity. Following those options, type `dna` or `protein` based on the dataset you want to analyse, then the name of a file containing a list of fasta files you would like to analyse, with one file on each line. Finally, type the name of the output prefix that you would like for each of the Gene-Wise analyses. This won't affect the names of the individual per-gene analyses (they will use to the name of the input fasta file), but it will give a unique prefix to the output files that analyse the whole dataset.

```
BOSTIn.GeneWise.sh [--blh] [--s] [--ch] <datatype> <text file containing a  
list of fasta files> <output file prefix>
```

Understanding The Outputs of Genewise

Genewise is a compilation program to run lots of BOSTIn analyses at once. This means that, broadly, genewise results can be interpreted in the same manner as a single gene analysis. For example, the Genewise histograms are histograms of the values that describe the entire gene for each gene in your input list, but they can be considered as broadly equivalent to the distributions of taxa-specific values for a single gene. This was one of the driving forces behind BOSTIn — picking metrics that are not only quick and possible to perform without phylogenetic analysis, but that also scaled from a single dataset to multiple datasets easily! Because of the, if you need to work out what the patterns of your data look like across your genes, check the "Understanding" section of the relevant metric above.